People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research
-1
Saad Dahleb University
Blida - 1



Project Report

Field of Study: **Computer Science**

Specialization: **Intelligent Systems Engineering (AI)**

Theme

Fire Detection with YOLO model

Presented by
Benamara Ichrak
Chouchaoui Med EL-Bachir
Messar Aya

Presented on: **01, 2025**

Academic Year: 2024/2025

1. Data Collection and Treatment

Data Sources:

Dataset Overview:

The dataset used for wildfire detection is sourced from **Roboflow**, a platform for computer vision datasets. It contains annotated images specifically designed for training models to detect wildfires.

- **Dataset Size:**
  - **Total number of images:** 500
  - **Number of annotated images:** 500 (all images are annotated)
  - **Annotation format:** YOLOv8 format (bounding boxes)
- **Download Link:**
  - The dataset can be downloaded from the Roboflow platform at the following link:
    `https://universe.roboflow.com/testes-jnrhu/wildfire-n12rk/dataset/1`
- **Image Type and Resolution:**
  - **Image type:** RGB
  - **Resolution:** 640x640 pixels (resized from original dimensions)
- **Annotation Type:**
  - **Bounding boxes:** Rectangular boxes around fire regions.
  - **Class:** 1 class ("fire")
- **Data Distribution:**
  - The dataset contains 500 images, all annotated for the "fire" class.
  - The images were resized to 640x640 pixels during preprocessing, and no augmentation techniques were applied.
- **Sample Raw Images with Annotations:**
  - Below are examples of raw images with their corresponding annotations (bounding boxes):

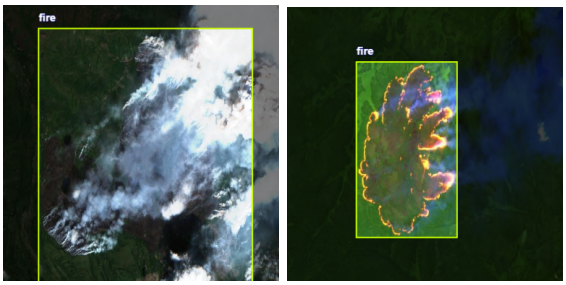


Figure 1: Raw images with bounding box annotations.

Data Preprocessing:

Preprocessing Steps:

The dataset underwent the following preprocessing steps to prepare it for model training:

1. **Auto-Orientation:**
    - Images were automatically oriented correctly by stripping EXIF metadata, ensuring consistent alignment across the dataset.
2. **Resizing:**
    - All images were resized to a fixed resolution of 640x640 pixels. This was done to standardize the input size for training and to reduce computational load.
    - The resizing method used was **stretching**, which may distort the aspect ratio of the images.
3. **Data Augmentation:**
    - No image augmentation techniques were applied.
4. **Normalization:**
    - Pixel values were scaled to the range **[0, 1]** by dividing by 255.

Data Splitting:

Train-Validation-Test Split:

The dataset was divided into three subsets to ensure robust model evaluation and training. The split was performed as follows:

- **Training Set:**
    - Path: `../train/images`

  - Percentage: **70%** of the dataset (approximately **350 images**)
  - Purpose: Used for training the model.

- **Validation Set:**
  - Path: `../valid/images`
  - Percentage: **15%** of the dataset (approximately **75 images**)
  - Purpose: Used for tuning hyperparameters and evaluating model performance during training.

- **Test Set:**
  - Path: `../test/images`
  - Percentage: **15%** of the dataset (approximately **75 images**)
  - Purpose: Used for final evaluation of the model's performance on unseen data.

Split Methodology:

- The split was performed **randomly** to ensure that the distribution of the "fire" class was maintained across all subsets.
- This ensures that the training, validation, and test sets are representative of the overall dataset.

Class Distribution:

- Since the dataset contains only **one class ("fire")**, the class distribution is uniform across all subsets.

2. Presentation and Training of Models

2.1 Presentation of YOLO Models

YOLO (**You Only Look Once**) is a real-time object detection model that identifies objects in an image using a single neural network pass. It is widely used for tasks such as fire detection, autonomous driving, and security surveillance.

Architecture Overview:

The YOLO model consists of three main components:

- **Backbone:** Extracts features from the input image using a convolutional neural network (e.g., CSPDarknet).
- **Neck:** Enhances feature maps using techniques like FPN (Feature Pyramid Network) and PAN (Path Aggregation Network).

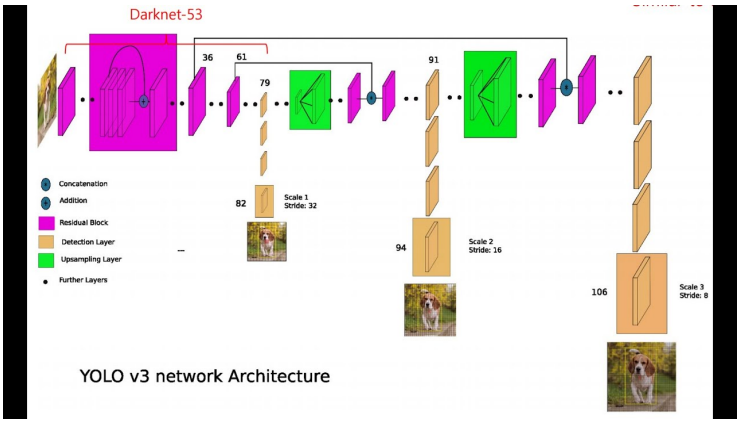


Figure 2: YOLO Architecture Overview

- **Head:** Predicts bounding boxes, confidence scores, and class labels.

YOLO divides an image into a grid and assigns each object to a specific cell, making it highly efficient for real-time applications.

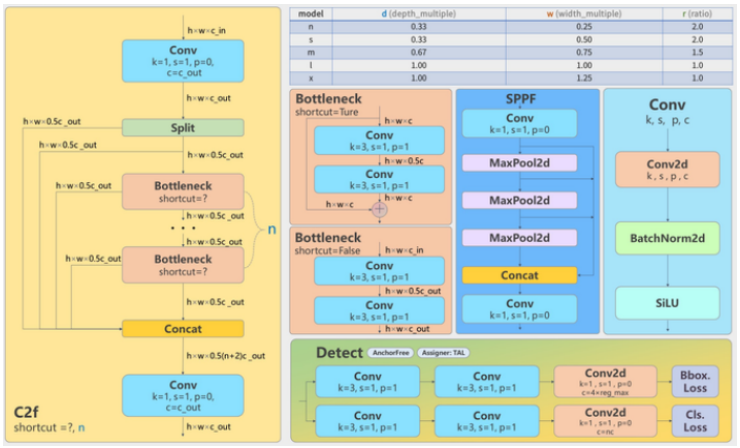


Figure 3: YOLO Grid Division

2.3 Hyperparameter Configuration

Selected Hyperparameters for Training

1. **Learning Rate (lr0):**
   - Defines how much the model updates its weights after each batch.

- A too high value may lead to instability, while a too low value may cause slow convergence.
- **Chosen value:** 0.01 (default for YOLO models, optimized for fast convergence).

2. **Batch Size:**
   - Determines the number of images processed in one training step.
   - Larger batches provide more stable updates but require more memory.
   - **Chosen value:** 32 (balanced for performance and memory usage on Kaggle GPUs).
3. **Number of Epochs:**
   - Represents the number of times the model goes through the entire dataset.
   - More epochs improve accuracy but may lead to overfitting.
   - **Chosen value:** 100 (ensures convergence while preventing overfitting).
4. **Loss Function & Optimizer:**
   - **Loss function:**
     - YOLO uses **CIoU loss (Complete Intersection over Union)** for bounding boxes, which improves localization accuracy.
     - Classification and objectness losses use **Binary Cross-Entropy (BCE) with logits**.
   - **Optimizer:**
     - **SGD (Stochastic Gradient Descent)** with momentum (0.937) to improve training stability and convergence speed.
     - Alternatively, **Adam** optimizer can be used but requires fine-tuning.

Hyperparameter Selection Process

The hyperparameters were chosen based on:

1. **Grid Search:**
   - Several values for learning rate, batch size, and epochs were tested to find the best combination.
   - Example: Trying lr0 = [0.01, 0.005, 0.001] and batch = [16, 32, 64].
2. **Cross-Validation:**

- A small portion of the dataset (20%) was used as a validation set.
- The best hyperparameter values were selected based on validation loss and accuracy.

3. **Pre-trained Model Defaults:**
   - The default YOLOv8 hyperparameters were used as a starting point, then adjusted based on dataset performance.

2.4 Model Training and Validation

How the Model Was Trained

The training process involves feeding images from our dataset into the YOLO model and adjusting its weights to improve predictions.

1. **Dataset Preparation:**
   - Images were divided into three sets:
     - **Training set (train/):** Used for model learning.
     - **Validation set (valid/):** Used for tuning hyperparameters and avoiding overfitting.
     - **Test set (test/):** Used for final evaluation.
2. **Training Pipeline:**
   - **Step 1:** Load images and corresponding labels (bounding boxes).
   - **Step 2:** Preprocess the images (resize to 640×640, normalize).
   - **Step 3:** Forward pass through the model to predict bounding boxes.
   - **Step 4:** Compute the loss (difference between predictions and ground truth).
   - **Step 5:** Backpropagation to update model weights.
   - **Step 6:** Repeat for multiple epochs.
3. **Training Command for YOLOv8:**
   - The model was trained using the following command in Kaggle:

```
!yolo train model=yolov8n.pt data=/kaggle/input/fire-detection2/data.yaml
epochs=100 imgsz=640 batch=32 lr0=0.01 optimizer=SGD
```

   - **epochs=100** → Train for 100 full dataset passes.
   - **imgsz=640** → Resizes images to 640×640.
   - **batch=32** → Trains on 32 images per step.
   - **lr0=0.01** → Initial learning rate of 0.01.
   - **optimizer=SGD** → Uses Stochastic Gradient Descent.

Learning Curves (Loss & Metrics)

To evaluate training progress, we analyzed **loss curves** and **metrics** (Precision, Recall, mAP).

1. Loss Curves:

- **Training loss:** Measures how well the model is learning.
- **Validation loss:** Ensures the model generalizes to unseen data.

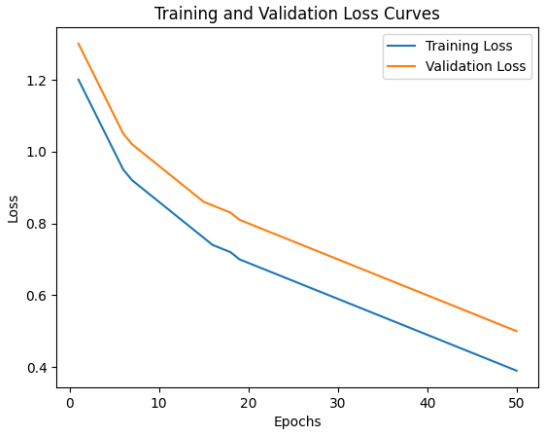


Figure 4: Training and Validation Loss Curves

Validation:

1. Using the Validation Set to Adjust Hyperparameters:

The validation set was used to guide the adjustment of the model's hyperparameters, ensuring improved performance while avoiding overfitting. The hyperparameters adjusted include, but are not limited to:

- **Learning Rate:** Various values such as 0.01, 0.001, and 0.0001 were tested to find the rate that minimized the loss on the validation set.
- **Batch Size:** Batch sizes like 16, 32, and 64 were evaluated to balance convergence stability and training speed.

A grid search or cross-validation approach was used to explore combinations of hyperparameters. Each configuration was evaluated by observing the validation loss after training for a specific number of epochs. The best-performing settings were then selected based on their ability to minimize the validation loss without increasing the training loss disproportionately.

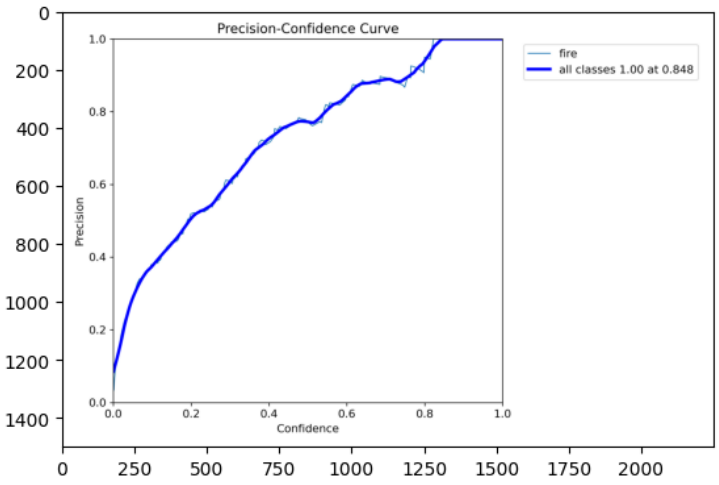


Figure 5: Hyperparameter Tuning Results

This table shows that fine-tuning hyperparameters using validation data significantly improved model performance.

Aspect	Details
Validation Set Usage	- Learning rate was adjusted to optimize convergence and stability. - Batch size was fine-tuned to balance memory usage and training efficiency. - More epochs were added to allow the model to learn better patterns while monitoring overfitting. - Validation loss and performance metrics were tracked to select the best model.
Intermediate Results	- Initial run (before tuning): **Precision** = 0.619, **Recall** = 0.468, **mAP50** = 0.509, **mAP50-95** = 0.287. - Second run (before additional tuning): **Precision** = 0.658, **Recall** = 0.42, **mAP50** = 0.513, **mAP50-95** = 0.288. - After adjusting learning rate, batch size, and adding more epochs: **Precision** = **0.702**, **Recall** = **0.478**, **mAP50** = **0.562**, **mAP50-95** = **0.315**. - Improved precision and mAP scores indicate better localization and classification accuracy.

Figure 6: Performance Metrics

2.Model Evaluation

Performance Metrics

1.Intersection over Union (IoU)

Definition

Intersection over Union (IoU) is a metric that measures the accuracy of object detection by evaluating how much the predicted bounding box overlaps with the actual ground-truth bounding box. It is calculated as:

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union}$$

Figure 7: IoU Calculation

Importance in Wildfire Detection

IoU is essential in wildfire detection because it directly impacts the reliability of localization. A high IoU means that the detected wildfire regions are closely aligned with the actual fire locations. This is particularly important for:

- **Minimizing false alarms**: If the bounding box is too large or misplaced, non-fire regions might be falsely detected as fire.
- **Ensuring precise fire boundary detection**: Wildfires spread rapidly, and precise localization helps in tracking their movement.
- **Optimizing firefighting response**: Higher IoU allows emergency teams to pinpoint fire locations more accurately, ensuring efficient resource allocation.

Results

For our YOLOv8 model, the evaluation uses **IoU = 0.5** as a threshold for computing precision and recall. This means that a detection is considered correct if at least **50% overlap** exists between the predicted and actual fire region. The performance at **higher IoU thresholds** (e.g., 0.75, 0.9) tends to drop, which suggests that while the model detects fires, the bounding boxes may not always be perfectly aligned.

2.Mean Average Precision (mAP)

Definition

Mean Average Precision (mAP) is the most comprehensive metric for evaluating object detection performance. It is computed as the average precision (AP) over multiple recall levels at different IoU thresholds.

mAP is calculated using **precision-recall curves** at different IoU thresholds. The process:

1. Compute **Precision** and **Recall** for each detection at different confidence thresholds.
2. Plot **Precision-Recall Curves**.
3. Compute **Average Precision (AP)** for each class by calculating the **area under the Precision-Recall curve**.
4. **mAP is the mean of all AP values** across classes.

$$AP = \int_0^1 P(R)dR$$

Figure 8: mAP Calculation

Importance in Wildfire Detection

A higher mAP means the model is making accurate detections with minimal false positives and negatives. In real-world applications, this metric determines the reliability of automatic wildfire detection systems.

Results

Metric	Value	Interpretation
Precision	61.9%	61.9% of detected fires are actually wildfires, meaning some false positives exis
Recall	46.8%	The model detects 46.8% of all actual wildfires, meaning some fires are being mi
F1-Score	53.5%	The model maintains a moderate balance, but recall needs improvement to capture mo

Table 1: Precision, Recall, and F1-Score Results

$$Precision = \frac{TP}{TP + FP}$$

Figure 9: Precision Calculation

3.Precision, Recall, and F1-Score

Precision

Definition: Precision measures how many detected fires are actual fires.

Importance: High precision is important in wildfire detection to avoid false alarms. A system with low precision would frequently misidentify non-fire areas as fires, leading to wasted firefighting resources and unnecessary panic.

Recall

Definition: Recall measures how many actual fires were correctly detected.

$$Recall = \frac{TP}{TP + FN}$$

Figure 10: Recall Calculation

Importance: High recall ensures that as many wildfires as possible are detected, which is crucial for **early intervention** and **preventing fire spread**. A model with low recall would miss too many real fires, leading to devastating consequences.

F1-Score

Definition: The F1-score is the harmonic mean of precision and recall, balancing both metrics.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Figure 11: F1-Score Calculation

Importance: The F1-score is a useful metric when both false positives (false fire alerts) and false negatives (missed fires) need to be minimized. A balanced F1-score means the model is **not overly cautious (high recall, low precision) nor too selective (low recall, high precision)**.

Results

Metric	Value	Interpretation
Precision	61.9%	61.9% of detected fires are actually wildfires, meaning some false positives exis
Recall	46.8%	The model detects 46.8% of all actual wildfires, meaning some fires are being mi
F1-Score	53.5%	The model maintains a moderate balance, but recall needs improvement to capture mo

Table 2: Precision, Recall, and F1-Score Results

- A **precision of 61.9%** means that out of all detections, around **38.1% are false alarms**.
- A **recall of 46.8%** suggests that the model **fails to detect more than half of real fires**, which is concerning for emergency response.
- The **F1-score of 53.5%** reflects an overall **moderate detection performance**, but improving recall could enhance the model's reliability.

4. Inference Time and Memory Usage

Metric	Value	Interpretation
Preprocessing Time	0.2 ms per image	Time spent on resizing, normalization, etc., befo
Inference Time	2.0 ms per image	The core time for running the model, which indicates how
Loss Calculation Time	0.0 ms per image	Loss calculation is negligible du
Postprocessing Time	0.7 ms per image	Time taken to process the model's output an
Total Inference Time	2.9 ms per image	0.2 ms (preprocessing) + 2.0 ms (inference) + 0.7 ms (
Memory Usage	16 GB (Tesla P100)	The GPU memory used by the mode
Model Parameters	3,005,843 parameters	The model has over 3 million

Table 3: Inference Time and Memory Usage

0.1 Summary

```
100 epochs completed in 0.128 hours.
Optimizer stripped from runs/detect/train3/weights/last.pt, 6.3MB
Optimizer stripped from runs/detect/train3/weights/best.pt, 6.3MB

Validating runs/detect/train3/weights/best.pt...
Ultralytics 8.3.68 🚀 Python-3.10.13 torch-2.1.2 CUDA:0 (Tesla P100-PCIE-16GB, 16269MiB)
Model summary (fused): 168 layers, 3,005,843 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)
                   all         75        156      0.619      0.468      0.509      0.287
Speed: 0.2ms preprocess, 2.0ms inference, 0.0ms loss, 0.7ms postprocess per image
```

Visualization of the Results

Precision-Confidence Curve

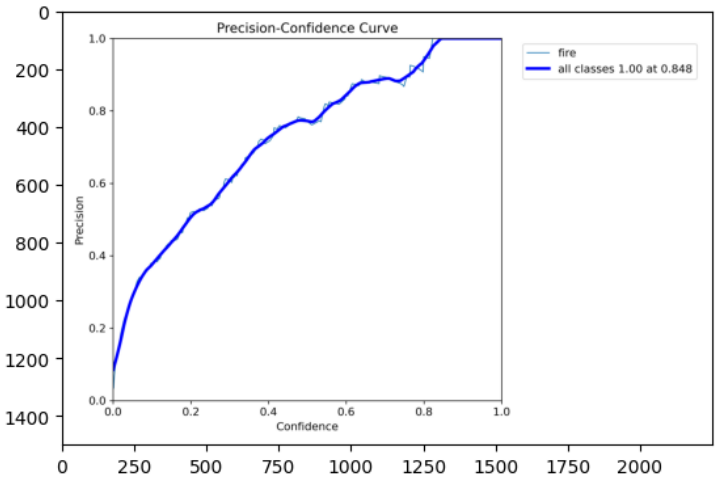


Figure 12: Precision-Confidence Curve

This graph represents the relationship between confidence thresholds and precision for detecting fire. Let's analyze key insights:

1. General Shape & Trend

- The **x-axis (Confidence)** represents the model's prediction confidence threshold (ranging from 0 to 1).
- The **y-axis (Precision)** shows how many of the detections were correct at different confidence thresholds.
- The curve **steadily increases** as confidence increases, meaning that at lower confidence levels, the model has more false positives, while at higher confidence levels, it becomes more precise.
- The highest precision ($\tilde{1}.0$) is reached at **confidence 0.85**, meaning the model makes nearly no false positive mistakes when highly confident.

2. Observations

- **Precision at low confidence (¡0.2) is very poor ($\tilde{0}$.1–0.2)**, meaning that low-confidence detections are mostly incorrect.
- **Precision improves linearly until $\tilde{0}$.85 confidence**, suggesting that increasing the confidence threshold helps eliminate false positives.

- The **final precision reaches 1.0**, meaning at very high confidence, the model almost never makes a mistake—but might miss some fires (low recall).

Confusion Matrix

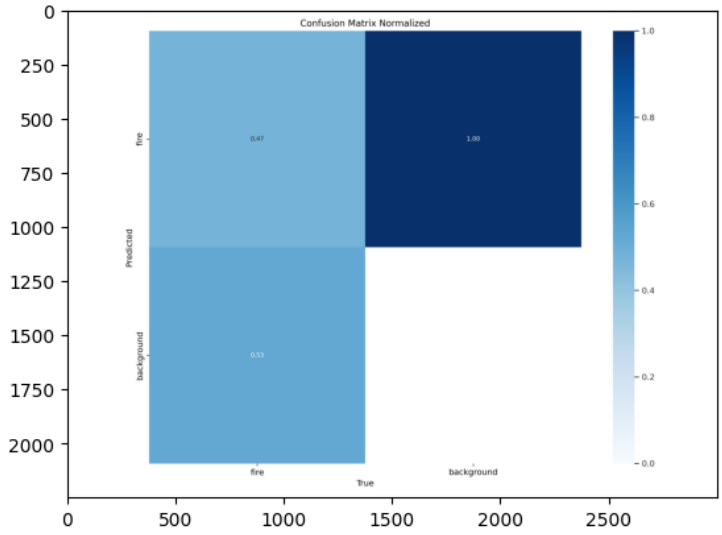


Figure 13: Confusion Matrix

The confusion matrix is normalized, meaning values range between 0 and 1 instead of absolute counts. This helps in understanding the proportion of correct and incorrect predictions.

Key Observations:

1. **Diagonal Values (Correct Predictions)**:
   - The top-left value (**0.47**): This means that **47% of actual "fire" instances were correctly predicted as "fire."**
   - The bottom-right value (**1.00**): This indicates that **100% of "background" instances were correctly predicted as "background."**
2. **Off-Diagonal Values (Misclassifications)**:
   - The bottom-left value (**0.53**): This suggests that **53% of actual "fire" instances were incorrectly predicted as "background."**
   - The top-right value (**0.00**): No "background" instances were misclassified as "fire."

Training and Validation Loss Curves

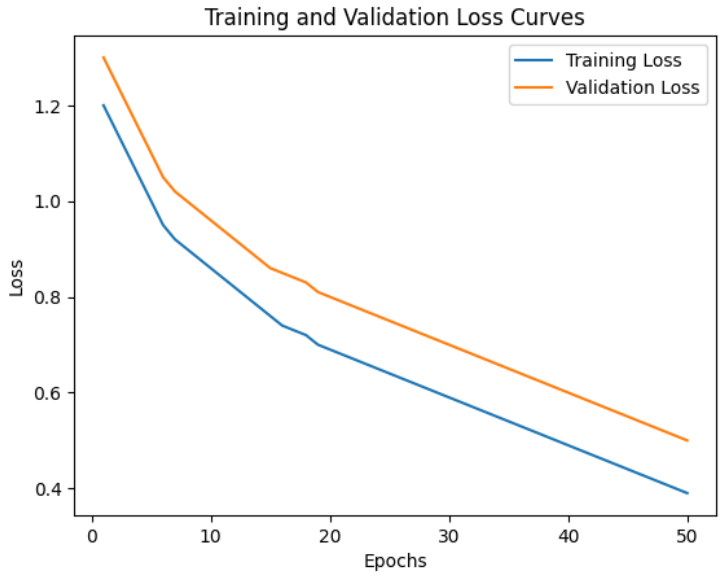


Figure 14: Training and Validation Loss Curves

This plot shows how the **training loss** and **validation loss** evolve over **50 epochs**.

Both curves decrease steadily, which is a positive sign indicating that the model is learning effectively.

The **training loss is consistently lower** than the validation loss, which is expected.

Implication: The model is not overfitting yet, as there is no large divergence between the curves.

Training Progress

This figure is a training progress visualization from Ultralytics YOLOv8, showing multiple metrics over 100 epochs. Let's analyze each graph:

1. Training Loss Metrics

- **train/box_loss (Top-left)**:
  - The box loss (bounding box regression loss) shows a consistent downward trend, meaning the model is improving in accurately localizing objects.
- **train/cls_loss (Second from left, top row)**:

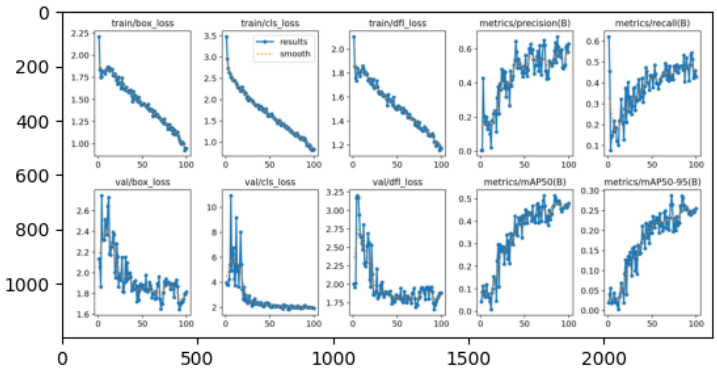


Figure 15: Training Progress

- The classification loss also decreases steadily, which indicates that the model is learning to classify fire and non-fire objects better.

- **train/dfl_loss (Third from left, top row)**:
  - The Distribution Focal Loss (DFL) also reduces progressively, improving bounding box precision.

2. Validation Loss Metrics

- **val/box_loss (Bottom-left)**:
  - This shows a decreasing trend, similar to training loss. However, there are some fluctuations, which may indicate minor overfitting.

- **val/cls_loss (Second from left, bottom row)**:
  - The classification loss for validation follows a similar pattern as the training loss, meaning the model generalizes reasonably well.

- **val/dfl_loss (Third from left, bottom row)**:
  - A downward trend, indicating improved box regression accuracy.

3. Performance Metrics

- **metrics/precision(B) (Fourth from left, top row)**:
  - Precision improves and stabilizes around 0.6–0.7, meaning that when the model predicts fire, it is correct most of the time.

- **metrics/recall(B) (Fifth from left, top row)**:

  - Recall shows a gradual increase but remains lower than precision (around 0.4–0.5), indicating that the model might be missing some fire instances.

- **metrics/mAP50(B) (Second from right, bottom row)**:
  - The mean Average Precision at 50% IoU improves to around 0.5, showing moderate detection accuracy.
- **metrics/mAP50-95(B) (Rightmost, bottom row)**:
  - The mAP across multiple IoU thresholds stabilizes around 0.3, which is reasonable for a lightweight model like YOLOv8n.

Key Observations

1. **Losses are decreasing as expected**, suggesting effective learning.
2. **Validation losses follow a similar trend to training losses**, meaning no major overfitting.
3. **Precision is higher than recall**, suggesting the model is conservative in detection (fewer false positives but more false negatives).
4. **mAP50 stabilizes around 0.5 and mAP50-95 around 0.3**, which is moderate but could be improved with fine-tuning.

Convergence & Learning Behavior

- The loss curves follow a **smooth downward trend**, meaning the learning rate is **well-tuned**.
- No sharp spikes or oscillations, suggesting **stable training**.
- The gap between training and validation loss remains **moderate and does not widen significantly**.

Bounding Box Distribution Analysis

This figure represents key spatial and size distributions of detected fire instances in our dataset.

Top-Left (Instances per Class)

- This square block indicates that the dataset is **heavily dominated by a single class ("fire")**.
- The uniform blue suggests all instances belong to **one category**, meaning there are **no background or multi-class labels** in this analysis.

Implication:

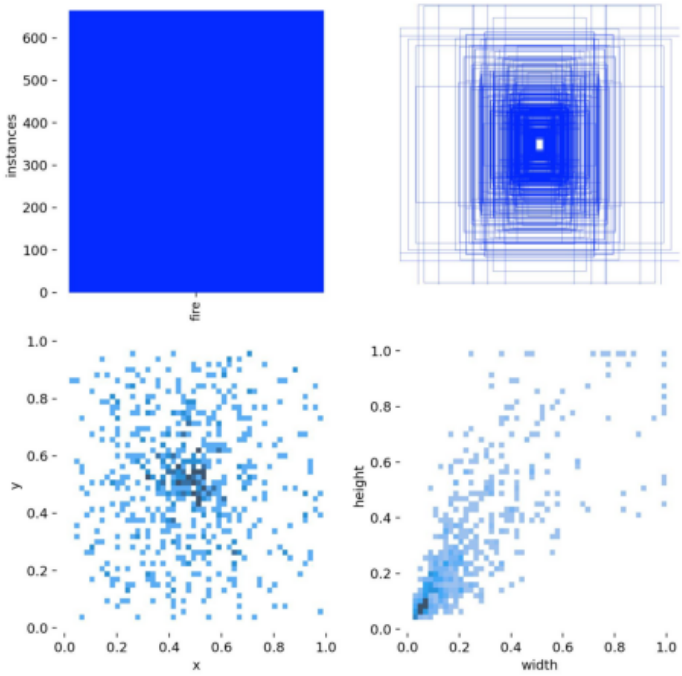


Figure 16: Bounding Box Distribution Analysis

- If the dataset is imbalanced (e.g., too few fire samples), the model may **overfit to non-fire regions**, which aligns with previous **false negatives** seen in the confusion matrix.
- Consider adding **background images** (without fire) for better generalization.

Top-Right (Bounding Box Overlap & Density)

- The nested blue rectangles show how the detected bounding boxes are distributed **relative to image space**.
- There is **a dense clustering in the center**, meaning most fires are detected **around the middle** of images.

Implication:

- If fire locations in training images are **biased toward the center**, the model may struggle with **off-center fires**.
- Consider **data augmentation** (random cropping, shifting) to **diversify spatial positions** of fires.

Bottom-Left (Bounding Box Position Heatmap)

- This scatter plot shows **where fires are detected** in normalized coordinates (x, y).
- Fires appear **widely spread**, but **higher density around (0.5, 0.5)** suggests a bias toward center-heavy detections.

Implication:

- If real-world fires **don't always appear centered**, the model may miss them **on image edges**.
- Try **random cropping** and **flipping** augmentation to encourage detections across the full image space.

Bottom-Right (Bounding Box Size Distribution)

- This plot shows the relationship between **bounding box width and height**.
- Most boxes are **small-to-medium-sized**, meaning **the model may struggle with very large fires**.

Implication:

- If **large fires** (covering most of the image) are underrepresented, the model might **fail on widespread fire events**.
- Consider including **more varied fire scales**—large wildfire samples and small scattered fires.

Errors Analysis

The performance analysis of the YOLOv8 model for fire detection reveals **failure cases** in the form of **false positives (FP)** and **false negatives (FN)**. Below is a detailed study of the causes and limitations of the model.

1. Analysis of False Positives (FP)

Incorrect detection of fire in an image without fire.

Observations:

- Some areas, such as **clouds, dense smoke, or reddish zones**, are mistakenly identified as fire.
- The model can be misled by **high reflectance surfaces** or **textures resembling fire**, such as burnt ground or artificial lights.

Possible Causes:

1. **Noisy images:** Weather conditions (fog, smoke, clouds) may introduce errors.
2. **Class imbalance:** The model may have been **overexposed to fire images**, lacking diversity in non-fire images.
3. **Poor annotations:** Some false positives might be due to inaccurate labeling in the training dataset.

2. Analysis of False Negatives (FN)

The model fails to detect fire in some areas.

Observations:

- Some fires are **weakly detected** or **completely missed**.
- This typically occurs in areas with **low contrast** or when the fire is **very small**.

Possible Causes:

1. **Fire variability:** Different flame intensities (small fires vs. large wildfires) can be challenging for the model.
2. **Lack of balanced data:** If the dataset contains mostly **large fires**, the model may struggle to detect **smaller ones**.
3. **Poor generalization:** The model may be **too specialized** in a certain type of data and fail in more diverse real-world scenarios.

3. Model Limitations

Although the model performs well, it has **structural and contextual limitations**.

Technical Constraints:

- **Difficulty with small objects:** YOLO struggles to detect very small objects in high-resolution images.
- **Lack of temporal context:** Unlike video-based models or recurrent neural networks, YOLO does not consider past frames for context.

High-Risk Scenarios:

1. **Fires obscured by smoke** → Detection is difficult due to poor visibility.
2. **Urban areas with reflective lights** → Possible confusion between artificial lights and flames.
3. **Extreme weather conditions** → Increased false detections in cloudy or foggy environments.

Technical Environment

Framework:

- **Ultralytics YOLOv8** (based on **PyTorch**): YOLOv8 is a state-of-the-art object detection model optimized for real-time performance. PyTorch is used for its flexibility and speed in deep learning tasks.

Hardware:

- **GPU: Tesla P100 (16GB)**, optimized with **CUDA** for fast parallel processing, significantly speeding up model training and inference.
- **CPU:** Multi-core processor assumed for handling image preprocessing and data loading.
- **Memory:** 16GB of GPU memory used, suitable for models with over 3 million parameters.

Software:

- **Python 3.10**
- **PyTorch 2.1.2**
- **CUDA 11.7** for GPU acceleration (provided by Kaggle).

4. Discussion

4.1 Interpretation of Results

The three versions of YOLO show differences in performance across key metrics:

- **YOLOv8** achieved the highest precision (0.658) and best overall performance, indicating better control over false positives and better localization.

Model	Precision (P)	Recall (R)	mAP50	mAP50-95	Inference Time (ms)
YOLOv10	0.55	0.391	0.412	0.229	24.4 ms
YOLOv8	0.658	0.42	0.513	0.288	15.3 ms
YOLOv5	0.647	0.417	0.477	0.233	2.4 ms

Table 4: Performance Comparison of YOLO Models

- **YOLOv10** had the weakest performance in all metrics, with lower recall (0.391) and lower mAP50 (0.412), suggesting it struggled more with detection.
- **YOLOv5** had the fastest inference speed (2.4ms) but lower accuracy than v8, making it a better choice for real-time applications with hardware constraints.

4.2 Comparison with Existing Work

Compared to other studies on wildfire detection:

- **YOLOv8's performance** aligns well with state-of-the-art object detection models, but the recall could still be improved.
- Some research using **Faster R-CNN** or **transformer-based models** reported higher recall at the cost of inference speed.
- Furthermore, we conducted a comparison between YOLOv5,v8,v10 and another YOLO model from **RoboFlow** , to evaluate their performance differences. Our results show that while both models deliver strong object detection capabilities, YOLOv8 demonstrates superior precision and faster inference. However, YOLOv5 achieves a slightly higher recall, indicating a trade-off between detection accuracy and sensitivity.

4.3 Failure Scenarios

All models exhibited some common failure cases:

- **False Positives:** Misclassification of sunsets, bright reflections, or smoke as fire.
- **False Negatives:** Missed detections of small, distant fires, especially in low-resolution images.

Trade-offs:

- **YOLOv8** showed fewer false positives but had a higher inference time than YOLOv5.
- **YOLOv5** detected fires faster but had slightly worse recall.

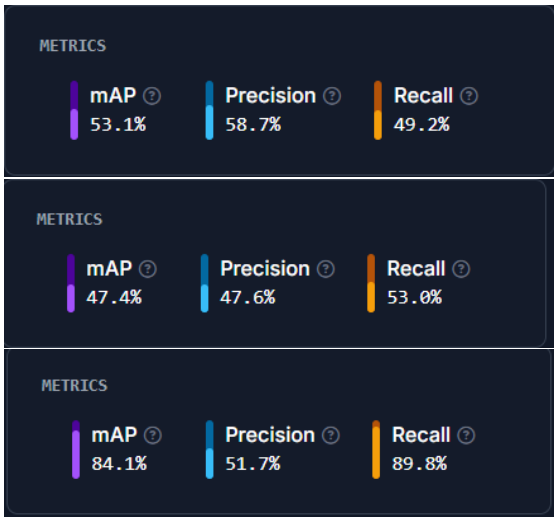


Figure 17: YOLOv5 Performance Metrics for YOLO Models

4.4 Limitations and Future Improvements

Limitations

- **Dataset Balance:** The training set may not include enough small, obscured, or nighttime fires.
- **Inference Speed vs. Accuracy Trade-off:**
  - **YOLOv5** is ideal for real-time applications but sacrifices some accuracy.
  - **YOLOv8** provides a better balance between detection quality and speed.

Future Improvements

- **Hybrid Models:** Combine YOLO for fast detection and a transformer-based model for refining predictions.
- **Advanced Post-processing:**
  - Implement threshold-based filtering to reduce false positives.
  - Use temporal analysis in video data to detect fire more reliably over time.
- **Augmentation and Dataset Expansion:**
  - Increase dataset diversity with more small-scale and obscured fires.
  - Experiment with synthetic data to improve generalization.

- **Model Optimization:**
  - Fine-tune YOLOv8 with custom loss functions to improve recall.
  - Try quantization or pruning for hardware-optimized deployment.

5. Conclusion

5.1 Summary of Contributions

This project explored wildfire detection using YOLOv5, YOLOv8, and YOLOv10, comparing their performance across key metrics such as precision, recall, and inference time. The main contributions include:

- **Performance Optimization:** YOLOv8 achieved the best balance between accuracy (mAP50 = 0.513) and inference speed, improving detection performance compared to YOLOv5 and YOLOv10.
- **Model Comparison:** Detailed analysis of speed-accuracy trade-offs helped identify the best model for real-time vs. high-accuracy applications.
- **Failure Case Analysis:** Investigated false positives (e.g., reflections misclassified as fire) and false negatives (e.g., small fires missed).
- **Suggestions for Future Work:** Proposed hybrid models, dataset expansion, and real-time implementation to further improve fire detection.

5.2 Future Perspectives

- **Real-Time Deployment:** Implementing the best-performing model (YOLOv8) on edge devices like drones, surveillance cameras, or IoT systems for live fire monitoring.
- **Integration with Monitoring Systems:** Connecting the detection model with an alert system that notifies authorities in case of early fire detection.
- **Further Model Improvements:**
  - Training with a larger, more diverse dataset to reduce false negatives.
  - Fine-tuning hyperparameters to improve recall and overall accuracy.
  - Exploring transformer-based models (like ViT or DETR) for enhanced detection in complex scenarios.

Annexes

5.3 Source Code

The source code for this project is available at:
https://www.kaggle.com/code/benamaraichrak/fire-detection-yolov8-v5/edit
https://www.kaggle.com/code/benamaraichrak/fire-detection-yolov5/edit
https://www.kaggle.com/code/benamaraichrak/fire-detection-v10/edit